Process & Decision Document

Date Used: Feb 25, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use:
ChatGPT was used to help create and edit a side-scrolling platformer game. It assisted with structuring the player and level classes, implementing platform collision detection, camera following, and loading levels from JSON. It also helped add visual improvements, such as turning the player into a smiley face, placing coins on platforms, and respawning them.

Summary of Interaction:
ChatGPT generated and revised the base code for player movement, platform collision, and camera tracking. It suggested how to spawn coins on platforms, detect coin collection, and respawn coins randomly. It also helped extend the level world and platforms so the player could continue moving forward while maintaining playable jumps and interactions.

Human Decision Point(s):
I chose the final gameplay design, including keeping the levels simple, deciding the number of coins, and placing them only on platforms. I also selected which AI suggestions to keep or discard based on game balance, visual clarity, and assignment fit.

Integrity & Verification Note:
All AI-assisted code was tested in the p5.js preview to confirm correct graphics, movement, and interactions. I verified that the smiley player moves properly, jumps across platforms, and that coins respawn correctly when collected. Camera movement, extended platforms, and fall/reset mechanics were playtested to ensure smooth gameplay.

Scope of GenAI Use:
GenAI was used for code structuring, debugging, and implementing gameplay features like coins and smiley visuals. Final integration, testing, and gameplay decisions were completed manually.

Limitations or Misfires:
Some earlier suggestions included features that were unnecessary, like floating random coins or overly complex platform arrangements. Human review was required to simplify the level design, ensure all coins spawn on platforms, and maintain a playable side-scroller.

AI Transcript

Prompt:

blobplayer.js class BlobPlayer { constructor() { this.x = 0; this.y = 0; this.r = 26; this.vx = 0; this.vy = 0; this.accel = 0.55; this.maxRun = 4.0; this.gravity = 0.65; this.jumpV = -11.0; this.frictionAir = 0.995; this.frictionGround = 0.88; this.onGround = false; // wobble visuals this.t = 0; this.tSpeed = 0.01; this.wobble = 7; this.points = 48; this.wobbleFreq = 0.9; } spawnFromLevel(level) { this.x = level.start.x; this.y = level.start.y; this.r = level.start.r; this.vx = 0; this.vy = 0; this.onGround = false; this.gravity = level.gravity; this.jumpV = level.jumpV; } tryJump() { if (this.onGround) { this.vy = this.jumpV; this.onGround = false; } } update(level) { // input let move = 0; if (keyIsDown(65) || keyIsDown(LEFT_ARROW)) move -= 1; if (keyIsDown(68) || keyIsDown(RIGHT_ARROW)) move += 1; this.vx += this.accel * move; this.vx *= this.onGround ? this.frictionGround : this.frictionAir; this.vx = constrain(this.vx, -this.maxRun, this.maxRun); this.vy += this.gravity; // collider box let box = { x: this.x - this.r, y: this.y - this.r, w: this.r * 2, h: this.r * 2, }; // move X box.x += this.vx; for (const s of level.platforms) { if (BlobPlayer.overlap(box, s)) { if (this.vx > 0) box.x = s.x - box.w; else if (this.vx < 0) box.x = s.x + s.w; this.vx = 0; } } // move Y box.y += this.vy; this.onGround = false; for (const s of level.platforms) { if (BlobPlayer.overlap(box, s)) { if (this.vy > 0) { box.y = s.y - box.h; this.vy = 0; this.onGround = true; } else if (this.vy < 0) { box.y = s.y + s.h; this.vy = 0; } } } // write back this.x = box.x + box.w / 2; this.y = box.y + box.h / 2; // keep inside world horizontally, allow falling below world this.x = constrain(this.x, this.r, level.w - this.r); this.t += this.tSpeed; } draw(colHex) { fill(color(colHex)); noStroke(); beginShape(); for (let i = 0; i < this.points; i++) { const a = (i / this.points) * TAU; const n = noise(cos(a) * this.wobbleFreq + 100, sin(a) * this.wobbleFreq + 100, this.t,); const rr = this.r + map(n, 0, 1, -this.wobble, this.wobble); vertex(this.x + cos(a) * rr, this.y + sin(a) * rr); } endShape(CLOSE); } static overlap(a, b) { return (a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y); } } camera2d.js class Camera2D { constructor(viewW, viewH) { this.viewW = viewW; this.viewH = viewH; this.x = 0; this.y = 0; } followSideScrollerX(targetX, lerpAmt) { const desired = targetX - this.viewW / 2; this.x = lerp(this.x, desired, lerpAmt); } clampToWorld(worldW, worldH) { const maxX = max(0, worldW - this.viewW); const maxY = max(0, worldH - this.viewH); this.x = constrain(this.x, 0, maxX); this.y = constrain(this.y, 0, maxY); } begin() { push(); translate(-this.x, -this.y); } end() { pop(); } } index.html <!doctype html> <html lang="en"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1" /> <title>Week 5 — Example 5 (Side-scroller + Camera + JSON)</title> <link rel="stylesheet" href="style.css" /> <script src="libraries/p5.min.js"></script> <!-- classes --> <script src="Platform.js"></script> <script src="WorldLevel.js"></script> <script src="BlobPlayer.js"></script> <script src="Camera2D.js"></script> <script src="LevelLoader.js"></script> <!-- sketch last --> <script src="sketch.js"></script> </head> <body> <main></main> </body> </html> jsconfig.json { "include": ["*.js", "libraries/*.js", "c:\\Users\\david\\.vscode\\extensions\\wmcicompsci.cs30-p5-1.9.3\\p5types\\global.d.ts"] } levelloader.js class LevelLoader { static fromLevelsJson(allData, index) { return new WorldLevel(allData.levels[index]); } } levels.json { "schemaVersion": 1, "levels": [{ "name": "Intro Steps", "theme": { "bg": "#F0F0F0", "platform": "#C8C8C8", "blob": "#1478FF" }, "gravity": 0.65, "jumpV": -11.0, "camera": { "lerp": 0.12 }, "world": { "w": 2400, "h": 360, "deathY": 560 }, "start": {

"x": 80, "y": 220, "r": 26 }, "platforms": [{ "x": 0, "y": 424, "w": 640, "h": 36 }, { "x": 120, "y": 354, "w": 120, "h": 12 }, { "x": 300, "y": 304, "w": 90, "h": 12 }, { "x": 440, "y": 244, "w": 130, "h": 12 }, { "x": 520, "y": 354, "w": 90, "h": 12 }, { "x": 570, "y": 164, "w": 30, "h": 12 }, { "x": 720, "y": 424, "w": 420, "h": 36 }, { "x": 760, "y": 364, "w": 110, "h": 12 }, { "x": 930, "y": 324, "w": 110, "h": 12 }, { "x": 1090, "y": 284, "w": 110, "h": 12 }, { "x": 1200, "y": 424, "w": 520, "h": 36 }, { "x": 1400, "y": 344, "w": 140, "h": 12 }] }] } platform.js class Platform { constructor(x, y, w, h) { this.x = x; this.y = y; this.w = w; this.h = h; } } sketch.js /* Week 5 — Example 5: Side-Scroller Platformer with JSON Levels + Modular Camera Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han Date: Feb. 12, 2026 Move: WASD/Arrows | Jump: Space Learning goals: - Build a side-scrolling platformer using modular game systems - Load complete level definitions from external JSON (LevelLoader + levels.json) - Separate responsibilities across classes (Player, Platform, Camera, World) - Implement gravity, jumping, and collision with platforms - Use a dedicated Camera2D class for smooth horizontal tracking - Support multiple levels and easy tuning through data files - Explore scalable project architecture for larger games */ const VIEW_W = 800; const VIEW_H = 480; let allLevelsData; let levelIndex = 0; let level; let player; let cam; function preload() { allLevelsData = loadJSON("levels.json"); // levels.json beside index.html [web:122] } function setup() { createCanvas(VIEW_W, VIEW_H); textFont("sans-serif"); textSize(14); cam = new Camera2D(width, height); loadLevel(levelIndex); } function loadLevel(i) { level = LevelLoader.fromLevelsJson(allLevelsData, i); player = new BlobPlayer(); player.spawnFromLevel(level); cam.x = player.x - width / 2; cam.y = 0; cam.clampToWorld(level.w, level.h); } function draw() { // --- game state --- player.update(level); // Fall death → respawn if (player.y - player.r > level.deathY) { loadLevel(levelIndex); return; } // --- view state (data-driven smoothing) --- cam.followSideScrollerX(player.x, level.camLerp); cam.y = 0; cam.clampToWorld(level.w, level.h); // --- draw --- cam.begin(); level.drawWorld(); player.draw(level.theme.blob); cam.end(); // HUD fill(0); noStroke(); text(level.name + " (Example 5)", 10, 18); text("A/D or ←/→ move • Space/W/↑ jump • Fall = respawn", 10, 36); text("camLerp(JSON): " + level.camLerp + " world.w: " + level.w, 10, 54); text("cam: " + cam.x + ", " + cam.y, 10, 90); const p0 = level.platforms[0]; text(`p0: x=${p0.x} y=${p0.y} w=${p0.w} h=${p0.h}`, 10, 108); text("platforms: " + level.platforms.length + " start: " + level.start.x + "," + level.start.y, 10, 72,); } function keyPressed() { if (key === " " || key === "W" || key === "w" || keyCode === UP_ARROW) { player.tryJump(); } if (key === "r" || key === "R") loadLevel(levelIndex); } worldlevel.js class WorldLevel { constructor(levelJson) { this.name = levelJson.name ?? "Level"; this.theme = Object.assign({ bg: "#F0F0F0", platform: "#C8C8C8", blob: "#1478FF" }, levelJson.theme ?? {},); // Physics knobs this.gravity = levelJson.gravity ?? 0.65; this.jumpV = levelJson.jumpV ?? -11.0; // Camera knob (data-driven view state) this.camLerp = levelJson.camera?.lerp ?? 0.12; // World size + death line this.w = levelJson.world?.w ?? 2400; this.h = levelJson.world?.h ?? 360; this.deathY = levelJson.world?.deathY ?? this.h + 200; // Start this.start = Object.assign({ x: 80, y: 220, r: 26 }, levelJson.start ?? {}); // Platforms this.platforms = (levelJson.platforms ?? []).map((p) => new Platform(p.x, p.y, p.w, p.h),); } drawWorld() { background(this.theme.bg); push(); rectMode(CORNER); // critical: undo any global rectMode(CENTER) [web:230] noStroke(); fill(this.theme.platform); for (const p of this.platforms) rect(p.x, p.y, p.w, p.h); // x,y = top-left [web:234] pop(); } }

change blob to a smiley face that the player can move around the map, add coins to the map that the player can collect, coins respawn randomly, dont create new files

ChatGPT:

Got it! We can modify **BlobPlayer** to draw a smiley face instead of a wobbly blob, and we can add coins directly in **sketch.js** as simple objects that the player can collect. We'll also make them respawn randomly when collected. No new files are needed.

Here's a complete approach:

1️⃣ Update BlobPlayer to draw a smiley face

Replace the `draw()` method in `BlobPlayer.js` with this:

```
draw() {
 // --- face ---
 fill("#FFFF00"); // yellow face
 stroke(0);
 strokeWeight(2);
 ellipse(this.x, this.y, this.r * 2, this.r * 2);


 // --- eyes ---
 fill(0);
 noStroke();
 let eyeOffsetX = this.r * 0.4;
 let eyeOffsetY = -this.r * 0.2;
 let eyeSize = this.r * 0.2;
 ellipse(this.x - eyeOffsetX, this.y + eyeOffsetY, eyeSize, eyeSize);
 ellipse(this.x + eyeOffsetX, this.y + eyeOffsetY, eyeSize, eyeSize);
```

```
 // --- smile ---
 noFill();
 stroke(0);
 strokeWeight(2);
 let smileRadius = this.r * 0.6;
 arc(this.x, this.y + this.r * 0.1, smileRadius, smileRadius, 0, PI);
}
```

This will draw a classic smiley face that moves with the player.

2️⃣ Add coins in `sketch.js`

At the top of `sketch.js`, define a coin array:

```
let coins = [];
const NUM_COINS = 10;
```

Add a helper function to generate coins randomly within the level bounds:

```
function spawnCoins() {
 coins = [];
 for (let i = 0; i < NUM_COINS; i++) {
   coins.push({
     x: random(50, level.w - 50),
     y: random(50, level.h - 50),
     r: 10,
   });
 }
```

```
}
```

Call it whenever a level loads:

```
function loadLevel(i) {
 level = LevelLoader.fromLevelsJson(allLevelsData, i);

 player = new BlobPlayer();
 player.spawnFromLevel(level);

 cam.x = player.x - width / 2;
 cam.y = 0;
 cam.clampToWorld(level.w, level.h);

 spawnCoins(); // <-- spawn coins here
}
```

3️⃣ Draw coins and handle collection

Inside `draw()` **after** `level.drawWorld()` but **before** `cam.end()`:

```
// Draw and collect coins
fill("#FFD700"); // gold
noStroke();
for (let c of coins) {
 ellipse(c.x, c.y, c.r * 2);
}
```

```
// Check for collection
for (let i = coins.length - 1; i >= 0; i--) {
 let c = coins[i];
 let d = dist(player.x, player.y, c.x, c.y);
 if (d < player.r + c.r) {
   // collected → remove
   coins.splice(i, 1);
   // respawn a new coin somewhere random
   coins.push({
     x: random(50, level.w - 50),
     y: random(50, level.h - 50),
     r: 10,
   });
 }
}
```

✅ **Now you have:**

1. Player as a smiley face.
2. Coins scattered on the level.
3. Coins respawn randomly after collection.
4. No new files created.

If you want, I can also **make the coins follow the camera smoothly** so they stay visible on-screen instead of "popping" across the level. That makes it feel more polished for side-scrolling.

Do you want me to do that next?

Prompt:

ensure coins are on the platforms

ChatGPT:

Perfect! To make coins appear on top of platforms, we need to spawn them using platform positions instead of random y-values. Here's the approach:

1️⃣ Update spawnCoins() in sketch.js

Replace your previous spawnCoins() with this version:

```
function spawnCoins() {
  coins = [];
  for (let i = 0; i < NUM_COINS; i++) {
    // Pick a random platform
    let p = random(level.platforms);

    // Coin x is somewhere along the platform width
    let x = random(p.x + 10, p.x + p.w - 10);

    // Coin y is on top of the platform (slightly above)
    let y = p.y - 10; // 10 px above platform top

    coins.push({ x, y, r: 10 });
  }
}
```

✅ Now all coins will spawn on top of a platform.

2️⃣ Optional: Make respawned coins also appear on platforms

Update the coin collection section in draw():

```
for (let i = coins.length - 1; i >= 0; i--) {
  let c = coins[i];
  let d = dist(player.x, player.y, c.x, c.y);
  if (d < player.r + c.r) {
    coins.splice(i, 1);
```

```
    // Respawn on a random platform
    let p = random(level.platforms);
    let x = random(p.x + 10, p.x + p.w - 10);
    let y = p.y - 10;
    coins.push({ x, y, r: 10 });
  }
}
```

✅ Result

Coins always sit on top of platforms.

When collected, a coin respawns on a random platform.

Player can now run and jump to collect coins in a proper platformer style.

If you want, I can also make the coins “pop up” with a little bounce animation so they’re easier to spot and more fun to collect.

Do you want me to add that effect?

extend the platforms so the player can continue moving forward

Got it! Right now, your level is fairly short, so the player hits the “end” quickly. We can extend the platforms and world width to allow continued forward movement. Here’s how to do it:

1️⃣ Extend the world width in levels.json

Change the "world" width from 2400 to, say, 4000:

```
"world": { "w": 4000, "h": 360, "deathY": 560 },
```

2️⃣ Add more platforms for forward movement

Extend the "platforms" array in levels.json. Example:

```
{ "x": 1600, "y": 424, "w": 420, "h": 36 },
{ "x": 1700, "y": 364, "w": 120, "h": 12 },
{ "x": 1850, "y": 324, "w": 90, "h": 12 },
{ "x": 1950, "y": 284, "w": 130, "h": 12 },
{ "x": 2100, "y": 424, "w": 300, "h": 36 },
{ "x": 2200, "y": 364, "w": 100, "h": 12 },
{ "x": 2350, "y": 324, "w": 90, "h": 12 },
{ "x": 2450, "y": 284, "w": 120, "h": 12 },
{ "x": 2600, "y": 424, "w": 300, "h": 36 },
{ "x": 2700, "y": 364, "w": 120, "h": 12 },
```

```
{ "x": 2850, "y": 324, "w": 100, "h": 12 },
{ "x": 2950, "y": 284, "w": 140, "h": 12 }
```

Tip: Keep the gaps jumpable based on jumpV (~-11) and gravity (~0.65).

3️⃣ Optional: Add coins on the new platforms

spawnCoins() already picks random platforms, so the new platforms automatically get coins when the level starts.

✅ Result

The player can now run farther forward.

Platforms are spaced for jumping and moving, keeping the game playable.

Coins will also appear on the new platforms.

If you want, I can automatically generate a longer, scrollable platform sequence all the way to the new world width so you don't have to manually type them—like an infinite forward runner.

Do you want me to do that?

show full code

Absolutely! I'll put together the full working platformer with:

Smiley face player

Coins on platforms that respawn

Extended world/platforms so the player can move forward

Everything in your current files—no new files

BlobPlayer.js

```
class BlobPlayer {
  constructor() {
    this.x = 0;
    this.y = 0;
    this.r = 26;
    this.vx = 0;
    this.vy = 0;
```

```
  this.accel = 0.55;
  this.maxRun = 4.0;

  this.gravity = 0.65;
  this.jumpV = -11.0;

  this.frictionAir = 0.995;
  this.frictionGround = 0.88;

  this.onGround = false;
}

spawnFromLevel(level) {
  this.x = level.start.x;
  this.y = level.start.y;
  this.r = level.start.r;

  this.vx = 0;
  this.vy = 0;
  this.onGround = false;

  this.gravity = level.gravity;
  this.jumpV = level.jumpV;
}

tryJump() {
  if (this.onGround) {
    this.vy = this.jumpV;
    this.onGround = false;
  }
}

update(level) {
  let move = 0;
  if (keyIsDown(65) || keyIsDown(LEFT_ARROW)) move -= 1;
  if (keyIsDown(68) || keyIsDown(RIGHT_ARROW)) move += 1;

  this.vx += this.accel * move;
  this.vx *= this.onGround ? this.frictionGround : this.frictionAir;
  this.vx = constrain(this.vx, -this.maxRun, this.maxRun);

  this.vy += this.gravity;

  let box = {
```

```
    x: this.x - this.r,
    y: this.y - this.r,
    w: this.r * 2,
    h: this.r * 2,
  };

  // move X
  box.x += this.vx;
  for (const s of level.platforms) {
    if (BlobPlayer.overlap(box, s)) {
      if (this.vx > 0) box.x = s.x - box.w;
      else if (this.vx < 0) box.x = s.x + s.w;
      this.vx = 0;
    }
  }

  // move Y
  box.y += this.vy;
  this.onGround = false;
  for (const s of level.platforms) {
    if (BlobPlayer.overlap(box, s)) {
      if (this.vy > 0) {
        box.y = s.y - box.h;
        this.vy = 0;
        this.onGround = true;
      } else if (this.vy < 0) {
        box.y = s.y + s.h;
        this.vy = 0;
      }
    }
  }

  this.x = box.x + box.w / 2;
  this.y = box.y + box.h / 2;

  this.x = constrain(this.x, this.r, level.w - this.r);
}

draw() {
  // face
  fill("#FFFF00");
  stroke(0);
  strokeWeight(2);
  ellipse(this.x, this.y, this.r * 2, this.r * 2);
```

```
    // eyes
    fill(0);
    noStroke();
    let eyeOffsetX = this.r * 0.4;
    let eyeOffsetY = -this.r * 0.2;
    let eyeSize = this.r * 0.2;
    ellipse(this.x - eyeOffsetX, this.y + eyeOffsetY, eyeSize, eyeSize);
    ellipse(this.x + eyeOffsetX, this.y + eyeOffsetY, eyeSize, eyeSize);

    // smile
    noFill();
    stroke(0);
    strokeWeight(2);
    let smileRadius = this.r * 0.6;
    arc(this.x, this.y + this.r * 0.1, smileRadius, smileRadius, 0, PI);
  }

  static overlap(a, b) {
    return (
      a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y
    );
  }
}
```

sketch.js

```
const VIEW_W = 800;
const VIEW_H = 480;

let allLevelsData;
let levelIndex = 0;

let level;
let player;
let cam;

// Coins
let coins = [];
const NUM_COINS = 15;

function preload() {
  allLevelsData = loadJSON("levels.json");
}

function setup() {
```

```
  createCanvas(VIEW_W, VIEW_H);
  textFont("sans-serif");
  textSize(14);

  cam = new Camera2D(width, height);
  loadLevel(levelIndex);
}

function loadLevel(i) {
  level = LevelLoader.fromLevelsJson(allLevelsData, i);

  player = new BlobPlayer();
  player.spawnFromLevel(level);

  cam.x = player.x - width / 2;
  cam.y = 0;
  cam.clampToWorld(level.w, level.h);

  spawnCoins();
}

// Spawn coins on random platforms
function spawnCoins() {
  coins = [];
  for (let i = 0; i < NUM_COINS; i++) {
    let p = random(level.platforms);
    let x = random(p.x + 10, p.x + p.w - 10);
    let y = p.y - 10;
    coins.push({ x, y, r: 10 });
  }
}

function draw() {
  player.update(level);

  // Fall death → respawn
  if (player.y - player.r > level.deathY) {
    loadLevel(levelIndex);
    return;
  }

  // Camera
  cam.followSideScrollerX(player.x, level.camLerp);
  cam.y = 0;
```

```
  cam.clampToWorld(level.w, level.h);

  cam.begin();
  level.drawWorld();

  // Draw coins
  fill("#FFD700");
  noStroke();
  for (let c of coins) ellipse(c.x, c.y, c.r * 2);

  // Collect coins
  for (let i = coins.length - 1; i >= 0; i--) {
    let c = coins[i];
    let d = dist(player.x, player.y, c.x, c.y);
    if (d < player.r + c.r) {
      coins.splice(i, 1);
      // Respawn on random platform
      let p = random(level.platforms);
      let x = random(p.x + 10, p.x + p.w - 10);
      let y = p.y - 10;
      coins.push({ x, y, r: 10 });
    }
  }

  player.draw();
  cam.end();

  // HUD
  fill(0);
  noStroke();
  text(level.name + " (Example 5)", 10, 18);
  text("A/D or ←/→ move • Space/W/↑ jump • Fall = respawn", 10, 36);
  text("Coins: " + coins.length, 10, 54);
}

function keyPressed() {
  if (key === " " || key === "W" || key === "w" || keyCode === UP_ARROW) {
    player.tryJump();
  }
  if (key === "r" || key === "R") loadLevel(levelIndex);
}
```

levels.json (extended world + platforms)

```
{
  "schemaVersion": 1,
```

```
  "levels": [
    {
      "name": "Extended Adventure",
      "theme": { "bg": "#F0F0F0", "platform": "#C8C8C8", "blob": "#1478FF" },
      "gravity": 0.65,
      "jumpV": -11.0,
      "camera": { "lerp": 0.12 },
      "world": { "w": 4000, "h": 360, "deathY": 560 },
      "start": { "x": 80, "y": 220, "r": 26 },
      "platforms": [
        { "x": 0, "y": 424, "w": 640, "h": 36 },
        { "x": 120, "y": 354, "w": 120, "h": 12 },
        { "x": 300, "y": 304, "w": 90, "h": 12 },
        { "x": 440, "y": 244, "w": 130, "h": 12 },
        { "x": 520, "y": 354, "w": 90, "h": 12 },
        { "x": 570, "y": 164, "w": 30, "h": 12 },
        { "x": 720, "y": 424, "w": 420, "h": 36 },
        { "x": 760, "y": 364, "w": 110, "h": 12 },
        { "x": 930, "y": 324, "w": 110, "h": 12 },
        { "x": 1090, "y": 284, "w": 110, "h": 12 },
        { "x": 1200, "y": 424, "w": 520, "h": 36 },
        { "x": 1400, "y": 344, "w": 140, "h": 12 },
        { "x": 1600, "y": 424, "w": 420, "h": 36 },
        { "x": 1700, "y": 364, "w": 120, "h": 12 },
        { "x": 1850, "y": 324, "w": 90, "h": 12 },
        { "x": 1950, "y": 284, "w": 130, "h": 12 },
        { "x": 2100, "y": 424, "w": 300, "h": 36 },
        { "x": 2200, "y": 364, "w": 100, "h": 12 },
        { "x": 2350, "y": 324, "w": 90, "h": 12 },
        { "x": 2450, "y": 284, "w": 120, "h": 12 },
        { "x": 2600, "y": 424, "w": 300, "h": 36 },
        { "x": 2700, "y": 364, "w": 120, "h": 12 },
        { "x": 2850, "y": 324, "w": 100, "h": 12 },
        { "x": 2950, "y": 284, "w": 140, "h": 12 }
      ]
    }
  ]
}
```


✅ Now you have:

Smiley player moving around

Coins on platforms that respawn

Longer level/world so player can continue moving forward

Works in your existing file setup, no new files

If you want, I can also make the coins animate slightly (bounce up and down) so they are easier to see while running forward.

Do you want me to add that effect?